🥞 स्टैक (Stack) - डेटा स्ट्रक्चर

1. परिभाषा (Definition)

स्टैक एक लीनियर डेटा स्ट्रक्चर (Linear Data Structure) है जो LIFO (Last In, First Out) सिद्धांत पर काम करता है।

- **LIFO का अर्थ: जो एलिमेंट सबसे अंत में डाला जाता है, वह सबसे पहले निकाला जाता है।**
- **उदाहरण: शादी में प्लेटों का ढेर (Stack of plates)। आप हमेशा सबसे ऊपर वाली प्लेट को सबसे पहले उठाते हैं।**

---

2. मुख्य ऑपरेशन्स (Key Operations)

स्टैक में मुख्य रूप से निम्नलिखित ऑपरेशन्स होते हैं:

Operation	कार्य (Function)	Python में कैसे करें?
Push	स्टैक में एक नया एलिमेंट जोड़ना (Add)।	`list.append(item)`
Pop	स्टैक से सबसे ऊपर वाला एलिमेंट हटाना (Remove)।	`list.pop()`
Peek / Top	बिना हटाए सबसे ऊपर वाले एलिमेंट को देखना।	`list[-1]`
IsEmpty	यह चेक करना कि स्टैक खाली है या नहीं।	`len(list) == 0`
Size	स्टैक में कुल एलिमेंट्स की संख्या पता करना।	`len(list)`

3. Python में इम्प्लीमेंटेशन (Implementation in Python)

Python में स्टैक को लागू करने का सबसे आसान तरीका List (लिस्ट) का उपयोग करना है।

A. सरल उदाहरण (Simple List Implementation)

Python code

```
# स्टैक बनाना (Creating a Stack)
stack = []

# 1. PUSH Operation (एलिमेंट डालना)
stack.append(10)
stack.append(20)
stack.append(30)

print("Push के बाद स्टैक:", stack)
# आउटपुट: [10, 20, 30]

# 2. PEEK Operation (टॉप एलिमेंट देखना)
top_element = stack[-1]
print("सबसे ऊपर वाला एलिमेंट:", top_element)
# आउटपुट: 30

# 3. POP Operation (एलिमेंट हटाना)
```

```
removed_element = stack.pop()
print("हटाया गया एलिमेंट:", removed_element)
print("Pop के बाद स्टैक:", stack)
# आउटपुट: [10, 20]
```

B. क्लास का उपयोग करके (Detailed Class Implementation)

परीक्षा या प्रोजेक्ट्स के लिए, स्टैक को एक Class के रूप में बनाना अधिक व्यवस्थित (structured) होता है।

Python code

```
class Stack:
    def __init__(self):
        self.items = []  # खाली लिस्ट

    # एलिमेंट को पुश करना
    def push(self, item):
        self.items.append(item)
        print(f"Pushed: {item}")

    # एलिमेंट को पॉप करना
    def pop(self):
        if not self.is_empty():
            return self.items.pop()
        else:
            return "Error: Stack Underflow (स्टैक खाली है)"

    # टॉप एलिमेंट देखना
    def peek(self):
        if not self.is_empty():
```

```
            return self.items[-1]
        else:
            return "Stack is Empty"


 # चेक करना कि स्टैक खाली है या नहीं
    def is_empty(self):
        return len(self.items) == 0

    # स्टैक का साइज
    def size(self):
        return len(self.items)

# --- उपयोग (Usage) ---
my_stack = Stack()

my_stack.push("Book A")
my_stack.push("Book B")
my_stack.push("Book C")

print("Top Book:", my_stack.peek())    # आउटपुट: Book C
my_stack.pop()                         # Book C हटेगा
print("Top Book Now:", my_stack.peek()) # आउटपुट: Book B
```

---

4. महत्वपूर्ण शर्तें (Important Terms)

1. **Stack Overflow (स्टैक ओवरफ्लो):** जब स्टैक पूरा भरा हो (fixed size stack में) और आप उसमें और एलिमेंट डालने की कोशिश करें। (Python list डायनामिक होती है, इसलिए इसमें यह समस्या कम आती है)।
2. **Stack Underflow (स्टैक अंडरफ्लो):** जब स्टैक खाली हो और आप उसमें से एलिमेंट हटाने (`pop`) की कोशिश करें।

---

5. स्टैक की समय जटिलता (Time Complexity)

Python List का उपयोग करते समय:

- **Push (append):** $O(1)$ (कांसटेंट टाइम)
- **Pop (pop):** $O(1)$ (कांसटेंट टाइम)
- **Peek (index -1):** $O(1)$ (कांसटेंट टाइम)

यह बहुत ही कुशल (Efficient) होता है।

📚 कतार (Queue) - डेटा स्ट्रक्चर

1. परिभाषा (Definition)

Queue (क्यू) एक लीनियर डेटा स्ट्रक्चर है जो FIFO (First In, First Out) सिद्धांत पर काम करता है।

- FIFO का अर्थ: जो एलिमेंट सबसे पहले डाला जाता है, वह सबसे पहले निकाला जाता है।
- उदाहरण: टिकट काउंटर पर लगी लाइन। जो व्यक्ति सबसे पहले लाइन में लगता है, उसे ही सबसे पहले टिकट मिलता है।

---

2. मुख्य ऑपरेशन्स (Key Operations)

Queue में मुख्य रूप से निम्नलिखित ऑपरेशन्स होते हैं:

Operation	कार्य (Function)	Python Terminology
Enqueue	कतार के अंत (Rear) में आइटम जोड़ना।	append()
Dequeue	कतार के सामने (Front) से आइटम हटाना।	popleft() (using deque)
Front / Peek	बिना हटाए सबसे आगे वाले एलिमेंट को देखना।	queue[0]
IsEmpty	यह चेक करना कि कतार खाली है या नहीं।	len(queue) == 0
Size	कतार की लंबाई पता करना।	len(queue)

---

3. Python में इम्प्लीमेंटेशन (Implementation in Python)

Python में Queue बनाने के लिए सामान्य list का उपयोग किया जा सकता है, लेकिन यह धीमा होता है (क्योंकि पहला एलिमेंट हटाने पर बाकी सबको शिफ्ट होना पड़ता है)।

इसलिए, हम collections मॉड्यूल से deque का उपयोग करते हैं, जो सबसे तेज और सही तरीका है।

A. Deque का उपयोग करके (Best Practice)

Python code

```
from collections import deque

# कतार बनाना (Creating a Queue)

queue = deque()


# 1. ENQUEUE Operation (एलिमेंट डालना)

queue.append("Person A")

queue.append("Person B")

queue.append("Person C")


print("Enqueue के बाद Queue:", queue)
```

आउटपुट: deque(['Person A', 'Person B', 'Person C'])

2. DEQUEUE Operation (एलिमेंट हटाना - जो पहले आया था)

first_person = queue.popleft() # popleft() सबसे बायीं (शुरुआती) तरफ से हटाता है

print("बाहर गया व्यक्ति:", first_person)

आउटपुट: Person A

print("अब Queue:", queue)

आउटपुट: deque(['Person B', 'Person C'])

---

4. Queue और Stack में अंतर (Difference)

विशेषता (Feature)	Stack (स्टैक)	Queue (कतार)
सिद्धांत	LIFO (Last In, First Out)	FIFO (First In, First Out)
निकालना (Removal)	सबसे बाद में आया एलिमेंट पहले निकलता है।	सबसे पहले आया एलिमेंट पहले निकलता है।
उपयोग (Use Case)	Undo बटन, Browser History	प्रिंटर की कतार, CPU शेड्यूलिंग

---

5. Queue की समय जटिलता (Time Complexity)

deque का उपयोग करते समय:

- **Enqueue (append): $O(1)$**
- **Dequeue (popleft): $O(1)$ (बहुत तेज)**
- ***(नोट: यदि आप साधारण list और pop(0) का उपयोग करते हैं, तो Dequeue $O(n)$ हो जाता है, जो धीमा है)***

Stack का अतिरिक्त उदाहरण (Extra Stack Example)

आपने Stack का एक और उदाहरण माँगा था। यहाँ एक बहुत ही लोकप्रिय उपयोग है: "स्ट्रिंग को उल्टा करना (Reverse a String)"।

चूंकि Stack LIFO (जो अंत में आया, वो पहले गया) है, अगर हम अक्षरों को स्टैक में डालें और फिर निकालें, तो वे उल्टे क्रम में निकलेंगे।

Python

```
def reverse_string(text):
    stack = []

    # 1. सारे अक्षरों को स्टैक में पुश करें (Push)
    for char in text:
        stack.append(char)

    reversed_text = ""

    # 2. एक-एक करके निकालें (Pop) और नई स्ट्रिंग बनाएं
```

```
    while len(stack) > 0:
        reversed_text += stack.pop()

    return reversed_text

# उपयोग (Usage)
original = "HELLO"
result = reverse_string(original)

print(f"असली शब्द: {original}")  # HELLO
print(f"उल्टा शब्द: {result}")    # OLLEH
```
